



UNIVERSIDADE FEDERAL DO PARANÁ
DEPARTAMENTO DE INFORMÁTICA
INTRODUÇÃO À COMPUTAÇÃO CIENTÍFICA

**OTIMIZAÇÃO DE DESEMPENHO PARA SISTEMAS
LINEARES ESPARSOS**

Relatório do Trabalho 2

Aluno: Pietro Comin (GRR: 20241955)

Curitiba
Dezembro de 2025

Resumo

Este relatório apresenta as técnicas de otimização aplicadas à resolução de sistemas lineares esparsos k-diagonais utilizando o Método dos Gradientes Conjugados. O estudo compara uma implementação ingênua baseada em matrizes densas (V1) com uma versão otimizada (V2) que utiliza o formato de armazenamento diagonal (DIA) e vetorização AVX. Os testes, realizados em processador AMD EPYC, demonstram que a mudança na estrutura de dados e no padrão de acesso à memória foi determinante para viabilizar a solução de grandes sistemas ($N = 20.000$), onde a versão original falhou por exaustão de recursos, atingindo um *speedup* superior a 6000x nas operações críticas.

1 Introdução

A eficiência na resolução de sistemas lineares $Ax = b$ é um pilar da computação científica. Em problemas reais, como simulações físicas, a matriz A frequentemente é esparsa (contém majoritariamente zeros) e estruturada em bandas (k -diagonal).

Neste contexto, o uso de estruturas de dados genéricas, como matrizes densas, acarreta dois problemas críticos: o desperdício massivo de memória RAM para armazenar zeros e o desperdício de ciclos de CPU realizando operações aritméticas nulas ($x \cdot 0$). Este trabalho visa superar essas limitações através da implementação de uma estrutura de armazenamento otimizada e do uso eficiente da hierarquia de memória e vetorização.

2 Metodologia e Modificações

2.1 Ambiente Experimental

Os experimentos foram conduzidos em uma servidora do Departamento de Informática (DInf) equipada com processador **AMD EPYC 7401 (Zen)** e compilador GCC com as flags de otimização `-O3 -march=native -mavx`. A análise de desempenho utilizou a ferramenta **LIKWID**.

2.2 Observação sobre a Versão 1 (V1)

Para viabilizar a comparação de desempenho em sistemas de grande porte ($N = 20.000$), foi necessário realizar uma pequena alteração na rotina de *geração* da matriz da Versão 1. A geração puramente ingênua ($O(N^3)$) levaria dias para concluir. A modificação restringiu-se apenas à inicialização dos dados; as estruturas de dados (matriz densa) e os algoritmos do solver (Gradiente Conjugado) permaneceram inalterados para preservar a fidelidade da comparação de performance do solver.

2.3 Limitações de Hardware

Devido à arquitetura do processador AMD utilizado, os grupos de performance referentes à Banda de Memória (L3/MEM) não estavam disponíveis no LIKWID. A análise de eficiência de memória baseou-se, portanto, na métrica de *Cache Miss Ratio*.

3 Estratégias de Otimização (V2)

A otimização concentrou-se na substituição da matriz densa pelo formato **DIA (Diagonal Storage Format)**. Em vez de alocar uma grade $N \times N$, a versão V2 armazena apenas as k diagonais não nulas em um vetor linearizado contíguo.

Fundamentação Teórica da Melhoria:

1. **Acesso Sequencial:** Ao armazenar diagonais sequencialmente, o algoritmo percorre a memória linearmente. Isso permite que o hardware antecipe os dados, reduzindo drasticamente as latências de memória (Cache Misses).
2. **Eliminação de 'Ifs' e Zeros:** A nova estrutura permite iterar apenas sobre os dados úteis. Remove-se a necessidade de testar se um elemento é zero, o que evita penalidades de previsão de desvio e elimina operações de ponto flutuante inúteis.
3. **Vetorização (AVX):** A contiguidade dos dados facilita a ação do compilador para gerar instruções SIMD (Single Instruction, Multiple Data), processando múltiplos elementos por ciclo de clock.

4 Estimativa de Desempenho

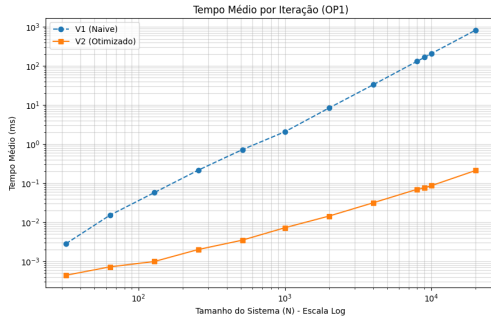
Considerando o cenário de $k = 7$ diagonais:

- **Capacidade de Resolução (Máximo N):** Na versão densa (V1), o consumo de memória cresce quadraticamente (N^2). Para 8GB de RAM, o limite teórico situa-se próximo a $N \approx 22.000$, mas overheads do sistema operacional causam falhas antes disso. Na versão otimizada (V2), o consumo cresce linearmente ($k \cdot N$). O limite de N torna-se virtualmente irrestrito para 8GB, sendo limitado apenas pelo tempo aceitável de execução.
- **Complexidade Computacional:** Uma iteração do solver na V1 realiza $2N^2$ operações, sendo a vasta maioria desperdício. Na V2, realizam-se apenas $\approx 14N$ operações (para $k = 7$). O ganho de tempo estimado é proporcional à razão N/k , indicando que quanto maior o sistema, maior a vantagem da V2.

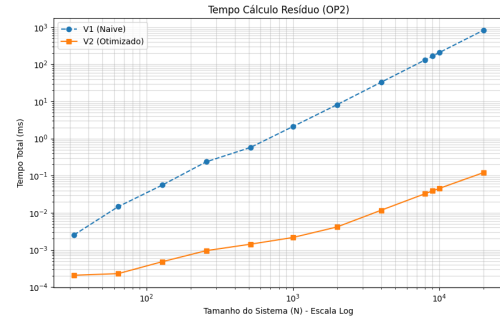
5 Análise de Resultados

5.1 Tempo de Execução e Speedup

Os gráficos abaixo (Fig. 1) comparam o tempo médio de execução. A escala logarítmica evidencia a disparidade de complexidade: a V1 cresce exponencialmente enquanto a V2 mantém comportamento linear.



(a) Tempo Médio por Iteração (Op1)



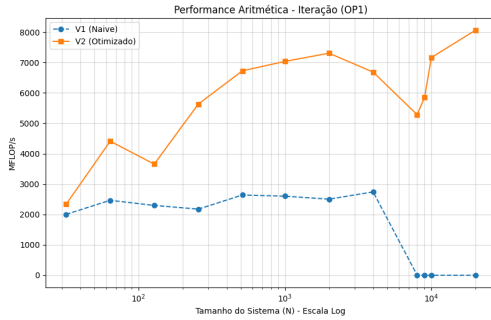
(b) Tempo Cálculo Resíduo (Op2)

Figura 1: Comparativo de Tempo (Escala Logarítmica). Note o crescimento acentuado da V1.

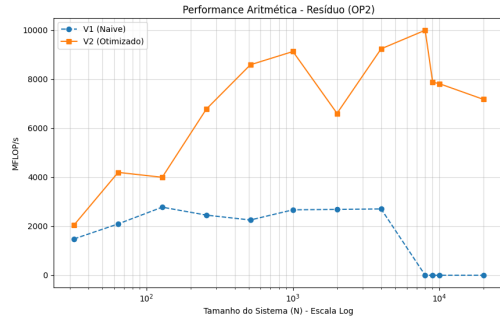
Para $N = 20.000$, a versão otimizada atingiu um **speedup de 3902x** na iteração e **6838x** no resíduo.

5.2 Desempenho Aritmético (FLOPS)

A Figura 2 ilustra a vazão de operações de ponto flutuante por segundo.



(a) FLOPS - Iteração (Op1)



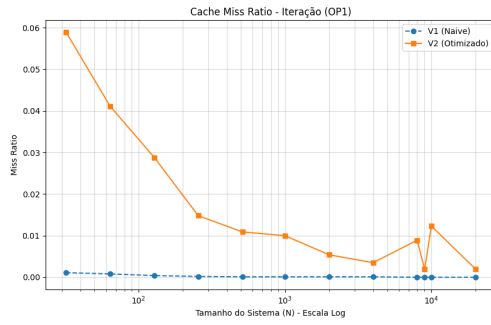
(b) FLOPS - Resíduo (Op2)

Figura 2: Performance Aritmética (DP MFLOP/s). A queda abrupta da V1 indica falha de execução.

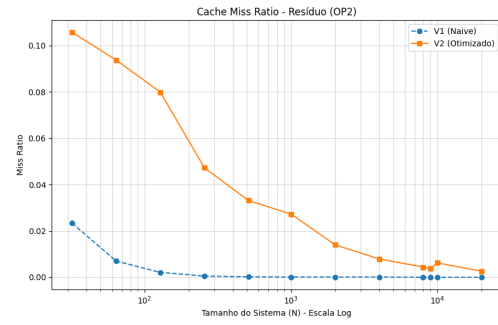
A V2 (laranja) escala eficientemente, atingindo picos próximos a 10.000 MFLOP/s, o que confirma o uso efetivo das unidades vetoriais (AVX). A V1 (azul), além de ter desempenho baixo, apresenta queda para zero em $N \geq 8000$. Isso ocorre porque a alocação de memória excessiva causou o encerramento prematuro do processo pelo sistema operacional durante a instrumentação, impedindo a coleta de métricas.

5.3 Eficiência de Cache

A eficiência do acesso à memória é demonstrada na Figura 3 através da taxa de Cache Miss.



(a) Cache Miss - Iteração (Op1)



(b) Cache Miss - Resíduo (Op2)

Figura 3: Taxa de Cache Miss (L1). A V2 mantém taxas consistentemente baixas.

A versão V2 apresentou taxas de miss extremamente baixas (frequentemente $< 1\%$), validando a hipótese de que o formato DIA favorece a localidade espacial e o prefetching.

6 Conclusão

As modificações implementadas transformaram o solver de sistemas lineares. A adoção do formato DIA eliminou o gargalo de memória, permitindo a resolução de sistemas ($N = 20.000$) que eram impossíveis para a versão ingênua. A combinação de acesso sequencial à memória e vetorização AVX resultou em ganhos de desempenho na ordem de milhares de vezes, comprovando a importância crítica da escolha adequada de estruturas de dados em Computação de Alto Desempenho.